Assignment 4 – Extra

Q1.

```
module priority_enc_tb;
logic  clk;
logic  rst;
logic  [3:0] D;
logic  [1:0] Y;
logic  valid;

priority_enc_no DUT (.*);

// Create the clock and reset
initial begin
   clk = 0;
   forever #1 clk = !clk;
end

// Create the Opcode and A and B
initial begin
   D = 0;
   assert_reset;

   D = 4'b1111;
   @(negedge clk);
   D = 4'b1110;
   @(negedge clk);
   D = 4'b1101;
   @(negedge clk);
   D = 4'b1100;
   @(negedge clk);
   D = 4'b1011;
   @(negedge clk);
   D = 4'b1010;
   @(negedge clk);
   D = 4'b1001;
   @(negedge clk);
   D = 4'b1000;
   @(negedge clk);
   D = 4'b0111;
   @(negedge clk);
   D = 4'b0110;
   @(negedge clk);
   D = 4'b0101;
   @(negedge clk);
   D = 4'b0100;
   @(negedge clk);
   D = 4'b0011;
   @(negedge clk);
   D = 4'b0010;
   @(negedge clk);
   D = 4'b0000;
   @(negedge clk);
   D = 4'b0001;
```

```
    D = 4'b0001;
    @(negedge clk);
    assert_reset;
    $stop;
end

task assert_reset;
    rst = 1;
    // Check reset value
    @(negedge clk);
    rst = 0;
endtask // assert_reset

property p_not_v;
 @(posedge clk) disable iff (rst) ($countones(D) == 0) |=> ~valid;
endproperty

property p_v;
 @(posedge clk) disable iff (rst) ($countones(D) > 0) |=> valid;
endproperty

property p_rst;
 @(posedge clk) (rst) |=> ~valid && ~Y;
endproperty

property p_Y_00;
 @(posedge clk) disable iff (rst) (D[3] == 1'b1 && D[2:0] == 3'b000) |=> (Y == 2'b00);
endproperty

property p_Y_01;
 @(posedge clk) disable iff (rst) (D[2] == 1'b1 && D[1:0] == 2'b00) |=> (Y == 2'b01);
endproperty

property p_Y_10;
 @(posedge clk) disable iff (rst) (D[1] == 1'b1 && D[0] == 1'b0) |=> (Y == 2'b10);
endproperty

property p_Y_11;
 @(posedge clk) disable iff (rst) (D[0] == 1'b1) |=> (Y == 2'b11);
endproperty

assert property (p_not_v);
cover property (p_not_v);

assert property (p_v);
cover property (p_v);

assert property (p_rst);
cover property (p_rst);
```

Kareem Waseem

```
assert property (p_not_v);
cover property (p_not_v);

assert property (p_v);
cover property (p_v);

assert property (p_rst);
cover property (p_rst);

assert property (p_Y_00);
cover property (p_Y_00);

assert property (p_Y_01);
cover property (p_Y_01);

assert property (p_Y_10);
cover property (p_Y_10);

assert property (p_Y_11);
cover property (p_Y_11);
endmodule
```

Kareem Waseem

```
module ALU_4_bit_tb;
// inputs to instantiated ALU
reg clk, reset;
reg  signed [3:0] A; // Input data A: 2's complement
reg  signed [3:0] B; // Input data B: 2's complement
// Output of ALU
wire signed [4:0] C; // ALU output: 2's complement

localparam Add = 2'b00,
       Sub = 2'b01,
       Not_A = 2'b10,
       ReductionOR_B = 2'b11;

localparam       MAXPOS = 7,
                 ZERO   = 0,
                 MAXNEG = -8;

reg [1:0] Opcode; // The opcode

ALU_4_bit DUT (.*);

// Create the clock and reset

initial begin
   clk = 0;
   forever #1 clk = !clk;
end

// Create the Opcode and A and B
initial begin
   A = 0;
   B = 0;
   assert_reset;

   // Apply all permutations of max pos, max neg, and 0 to each opcode
   A=MAXNEG; B=MAXNEG;
   Opcode = Add;
   @(negedge clk);
   Opcode = Sub;
   @(negedge clk);

   A=MAXNEG; B=ZERO;
   Opcode = Add;
   @(negedge clk);
   Opcode = Sub;
   @(negedge clk);
```

Kareem Waseem

```
A=MAXNEG; B=MAXPOS;
Opcode = Add;
@(negedge clk);
Opcode = Sub;
@(negedge clk);

A=ZERO; B=MAXNEG;
Opcode = Add;
@(negedge clk);
Opcode = Sub;
@(negedge clk);

A=ZERO; B=ZERO;
Opcode = Add;
@(negedge clk);
Opcode = Sub;
@(negedge clk);

A=ZERO; B=MAXPOS;
Opcode = Add;
@(negedge clk);
Opcode = Sub;
@(negedge clk);

A=MAXPOS; B=MAXNEG;
Opcode = Add;
@(negedge clk);
Opcode = Sub;
@(negedge clk);

A=MAXPOS; B=ZERO;
Opcode = Add;
@(negedge clk);
Opcode = Sub;
@(negedge clk);

A=MAXPOS; B=MAXPOS;
Opcode = Add;
@(negedge clk);
Opcode = Sub;
@(negedge clk);

// Set B to non-all 0 and non-all 1.
Opcode = Not_A;

A=4'b0000; B=4'b0101;
@(negedge clk);
A=4'b1111; B=4'b1010;
@(negedge clk);
```

Karim Waseem

```
    // Apply 0, all 1's, and walking 1's to B for ReductionOR_B opcode. Set A to a value that will yield the opposite result.
    Opcode = ReductionOR_B;

    A=4'b1111; B=4'b0000;
    @(negedge clk);
    A=4'b0000; B=4'b1111;
    @(negedge clk);
    A=4'b0000; B=4'b0001;
    @(negedge clk);
    A=4'b0000; B=4'b0010;
    @(negedge clk);
    A=4'b0000; B=4'b0100;
    @(negedge clk);
    A=4'b0000; B=4'b1000;
    @(negedge clk);

    // Set the operands and walk through the opcodes
    A=4'b0101; B=4'b1010;
    Opcode = Add;
    @(negedge clk);
    Opcode = Sub;
    @(negedge clk);
    Opcode = Not_A;
    @(negedge clk);
    Opcode = ReductionOR_B;
    @(negedge clk);
    $stop;
end

task assert_reset;
    reset = 1;
    @(negedge clk)
    reset = 0;
endtask // assert_reset

property p_ADD;
 @(posedge clk) disable iff (reset) (Opcode == Add) |=> (C == $past(A) + $past(B));
endproperty

property p_SUB;
 @(posedge clk) disable iff (reset) (Opcode == Sub) |=> (C == $past(A) - $past(B));
endproperty

property p_NOT_A;
 @(posedge clk) disable iff (reset) (Opcode == Not_A) |=> (C == ~$past(A));
endproperty

property p_RED_B;
 @(posedge clk) disable iff (reset) (Opcode == ReductionOR_B) |=> (C == |$past(B));
endproperty
```

```
assert property (p_ADD);
cover property (p_ADD);

assert property (p_SUB);
cover property (p_SUB);

assert property (p_NOT_A);
cover property (p_NOT_A);

assert property (p_RED_B);
cover property (p_RED_B);

always_comb begin
  if(reset)
    a_reset: assert final(C == 0);
end

endmodule
```

Kareem Waseem

Q2.

Write SVA properties for an arbiter design for the following specs:

1- Upon rising of the signal “request” by a master, the arbiter should raise the “grant” within 2 to 5 clock cycles.

```
property p_req_gnt;

@(posedge clk) $rose (req) |-> ##[2:5] $rose (gnt);

endproperty
```

2- Once the “grant” is raised, the master should acknowledge acceptance in the same clock cycle by lowering the "frame" and "irdy" signals after they were high in the previous cycle.

```
property p_gnt_frame;

@(posedge clk) $rose (grant) |-> $fell (frame) && $fell(irdy);

endproperty
```

3- Once the master completes the transaction it raises the "frame" and "irdy" signals, followed by that, the arbiter should lower the "grant" signal on the next clock cycle after it was high.

```
property p_frame_gnt;

@(posedge clk) $rose(frame) && $rose(irdy) |=> $fell(grant);

endproerty
```

Q3. Suppose you are a design engineer and would like to write assertions to the internals of your design that has an FSM, write down the following SVA properties as per the following specs:

1- Current state internal signal “cs” will always stay one-hot irrespective of the input conditions.

```
property p_onehot;

@(posedge clk) ($countones(cs) == 1 );

endproperty
```

2- If the current state “cs” is "IDLE" and if "get_data" input is raised, then the state is "GEN_BLK_ADDR" the following cycle and 64 cycles later the state should change to "WAITO."

```
property p_trans;

@(posedge clk) $rose(get_data) && (cs == IDLE) |=> (cs == GEN_BLK_ADDR) [*64] ##1 (cs == WAITO);

endproperty
```

Kareem Waseem